TURBINE-X

Research Project Execution Blueprint

Joint Conventional Machine Learning for Gas-Turbine Emission and Energy-Yield Estimation
with Uncertainty-Aware Historical Operating-Point Analysis

Dataset: UCI Gas Turbine CO and NOx Emission Data Set
Five uploaded annual CSV files • 36,733 observations • 2011–2015

Prepared for: B.Tech Computer Science Engineering
Development: Windows 11 • VS Code • Jupyter Notebook
Hardware: Ryzen 9 8940HX • RTX 5060 8 GB • 16 GB RAM

Version 1.0 | 4 November 2026

Document Control

Field	Value
Project	TURBINE-X
Document	Research Project Execution Blueprint and Dataset Audit
Version	1.0 — dataset-driven plan
Dataset status	All five user-uploaded CSV files physically inspected
Implementation status	Not started; starter scaffold only
Primary ML constraint	Conventional ML only; deep learning prohibited
Research status	Provisional; novelty requires further systematic literature search
Prepared date	4 November 2026

Important status distinction: the uploaded CSV audit is complete for the five files supplied in this conversation. The starter package contains notebook templates, a progress tracker, configuration and diagrams; it does not contain trained models or a finished application.

Table of Contents

- 1. Executive Summary
- 2. Project Identity and Research Scope
- 3. Dataset Inventory and File-Level Audit
- 4. Data Quality, Correlation and Leakage Assessment
- 5. Valid ML Problem and Target Formulations
- 6. Literature Review and Research Gap
- 7. Frozen Research Questions and Hypotheses
- 8. Frozen Experimental Protocol
- 9. Conventional ML Algorithm Selection
- 10. Feature Engineering and Preprocessing
- 11. Complete ML Pipeline
- 12. Project Folder Structure
- 13. Jupyter Notebook Execution Roadmap
- 14. Centralized Automatic Progress Tracker
- 15. Experiment and Output Management
- 16. Architecture Diagrams
- 17. Week-by-Week Implementation Schedule
- 18. Evaluation and Statistical Analysis
- 19. Testing and Quality Assurance
- 20. Application Design
- 21. Hardware, Software and Environment
- 22. Risk Register and Mitigations
- 23. Expected Deliverables and Acceptance Criteria
- 24. Final Supervisor Verdict
- 25. References

In Microsoft Word, update the automatic Table of Contents field if a dynamic TOC is preferred. This document also includes the section list above for immediate navigation.

1. Executive Summary

TURBINE-X is a conventional-machine-learning research project built around five original annual CSV files from the UCI Gas Turbine CO and NOx Emission Data Set. All five uploaded files were read directly. They contain 36,733 observations in total and share the same 11 numerical columns. The proposed project studies the estimation of two pollutant concentrations (CO and NOX) and turbine energy yield (TEY) from eight contemporaneous ambient and turbine measurements.

The central research comparison is between independent target models and genuinely joint multi-output models. Experiment A reproduces the familiar two-emission prediction formulation using TEY among the predictors. Experiment B is the main investigation: eight measured inputs predict CO, NOX and TEY jointly. This separation avoids the logical error of using TEY as an input while simultaneously claiming to predict or optimize TEY.

The data are annual and ordered chronologically, but the uploaded rows do not contain timestamps. The project must not infer exact time intervals, construct hourly lags, or claim a verified forecast horizon. The dataset is observational and from one plant; no causal or safe-control claims are supported. A historical operating-point trade-off explorer may be demonstrated, but unconstrained operational optimization is not justified by the available variables.

A 2026 paper already studies the same dataset with chronological development/calibration/test partitions, pollutant-specific models, temporal shift and high-emission tail risk. Consequently, TURBINE-X must not claim novelty from those elements alone. Its provisional research direction is the controlled comparison of independent versus joint multi-output conventional ML for CO, NOX and TEY, together with multi-output uncertainty assessment and support-aware descriptive trade-off analysis. The novelty claim remains provisional until a systematic literature review confirms that the combination has not already been addressed.

2. Project Identity and Research Scope

Item	Definition
Title	TURBINE-X: Joint Conventional ML for Gas-Turbine Emission and Energy-Yield Estimation
Domain	Industrial energy systems and environmental monitoring
Problem	Estimate contemporaneous CO, NOX and TEY from measured ambient and turbine operating variables
Primary task	Multi-output regression, compared with independent regression
Users	Researchers, plant analytics teams and academic evaluators; not turbine controllers
Aim	Assess whether joint output learning improves predictive usefulness and uncertainty characterization across annual operating domains
Scope	Offline analysis, reproducible model experiments, explanatory analysis and a demonstration dashboard
Out of scope	Causal inference, control recommendations, safety certification, regulatory compliance, real-time streaming and hourly-ahead forecasting

2.1 Objectives

1. Audit and document the five annual CSV files, including file hashes, schemas, row counts and quality checks.

2. Quantify annual target distributions, feature drift, correlations and predictor redundancy.
3. Establish two non-confounded prediction formulations with explicit feature/target separation.
4. Compare statistical baselines, tree ensembles, boosting and joint multi-output estimators.
5. Evaluate AutoML and hyperparameter optimization under fixed chronological validation and compute budgets.
6. Assess uncertainty intervals, calibration, residual behavior and generalization to the final year.
7. Explain model behavior using permutation importance, SHAP and dependence analysis, with correlation caveats.
8. Build a support-aware historical operating-point comparison interface and document limitations.

3. Dataset Inventory and File-Level Audit

Source: UCI Machine Learning Repository, Gas Turbine CO and NOx Emission Data Set, DOI 10.24432/C5WC95, CC BY 4.0. The five user-supplied files were read with pandas read_csv. File sizes below are exact byte counts measured in the uploaded files.

File	Rows	Columns	Bytes	Missing	Exact duplicate rows
gt_2011.csv	7411	11	562921	0	0
gt_2012.csv	7628	11	577617	0	0
gt_2013.csv	7152	11	540334	0	0
gt_2014.csv	7158	11	541009	0	7
gt_2015.csv	7384	11	556928	0	0
TOTAL	36733	11	2,778,809	0	7

All five files have identical headers and column order: AT, AP, AH, AFDP, GTEP, TIT, TAT, TEY, CDP, CO, NOX. Every column was parsed as float64. There are no categorical columns, explicit date/time columns, row identifiers or ID-like columns. The annual identity is encoded by the filename and must be attached as metadata for ordering and partitioning only.

3.1 Column dictionary

Column	Meaning	Unit	Experiment A	Experiment B
AT	Ambient temperature	°C	Input	Input
AP	Ambient pressure	mbar	Input	Input
AH	Ambient humidity	%	Input	Input
AFDP	Air filter differential pressure	mbar	Input	Input
GTEP	Gas turbine exhaust pressure	mbar	Input	Input
TIT	Turbine inlet temperature	°C	Input	Input
TAT	Turbine after temperature	°C	Input	Input
TEY	Turbine energy yield	MWh	Input	Target
CDP	Compressor discharge pressure	mbar	Input	Input
CO	Carbon monoxide	mg/m³	Target	Target
NOX	Nitrogen oxides	mg/m³	Target	Target

3.2 Actual sample record

The first record in gt_2011.csv is: AT=4.5878, AP=1018.7, AH=83.675, AFDP=3.5758, GTEP=23.979, TIT=1086.2, TAT=549.83, TEY=134.67, CDP=11.898, CO=0.32663, NOX=81.952. This row is reproduced from the uploaded original file.

4. Data Quality, Correlation and Leakage Assessment

Check	Measured result	Interpretation
Missing values	0 across all 11 columns	No imputation required for current snapshot; retain assertion tests
Non-finite values	0	No NaN or ±infinity observed
Exact duplicates	7 duplicate occurrences in 2014; 2 repeated-value groups	Retain primary data; run duplicate-excluded sensitivity analysis
Cross-year duplicates	0 identical physical rows across years	No exact physical-row overlap across annual files
Constant columns	None	All 11 columns vary
Data types	11 float64 per file	All columns numeric
Categorical/date/ID columns	None	Year comes from filename; no row timestamp
Single-site coverage	One gas turbine in northwestern Turkey per UCI description	External plant generalization cannot be established

4.1 Observed ranges

Variable	Minimum	Maximum
AT	-6.2348	37.103
AP	985.85	1036.6
AH	24.085	100.2
AFDP	2.0874	7.6106
GTEP	17.698	40.716
TIT	1000.8	1100.9
TAT	511.04	550.61
TEY	100.02	179.5
CDP	9.8518	15.159
CO	0.00038751	44.103
NOX	25.905	119.91

4.2 Correlation diagnostics

Feature pair	Pearson r	Risk
TEY–CDP	0.989	Very high redundancy
GTEP–CDP	0.978	Very high redundancy
GTEP–TEY	0.964	Very high redundancy
TIT–TEY	0.910	High redundancy
TIT–CDP	0.908	High redundancy
TIT–GTEP	0.874	High redundancy
AFDP–TIT	0.691	Moderate/high association

These correlations are descriptive, not causal. Correlated predictors can destabilize linear coefficients and split or permutation importance. Report grouped ablations and compare importance methods. Do not drop a variable solely because its pairwise correlation is high.

4.3 Leakage and validity controls

- Never use CO or NOX as predictors for the other emission target in the primary benchmark.
- Experiment B excludes TEY from inputs because TEY is a target there.
- Do not use filename-derived year as a predictive feature in the primary models; use it for split metadata only.
- Do not construct lag, rolling or lead variables because exact row timestamps and interval continuity are not available.
- Fit scaling, imputation (if later needed), feature selection and tuning only within development folds.
- Keep 2015 untouched until final evaluation; do not use it for model or threshold selection.
- Do not infer causality or safe operating setpoints from observational associations.

5. Valid ML Problem and Target Formulations

5.1 Experiment A — emission benchmark

Inputs: AT, AP, AH, AFDP, GTEP, TIT, TAT, TEY, CDP (nine features). Targets: CO and NOX. This formulation matches the common predictive-emission-monitoring use case and allows comparison with prior work. It is a benchmark, not the primary novelty claim.

5.2 Experiment B — primary joint-output investigation

Inputs: AT, AP, AH, AFDP, GTEP, TIT, TAT, CDP (eight features). Targets: CO, NOX and TEY. Compare independent per-target models with true multi-output estimators. This formulation avoids providing TEY to the model when TEY itself is being predicted.

5.3 Decision-support boundary

A dashboard may show predicted CO, NOX and TEY with intervals and compare observed or support-near historical operating points. Candidate search must remain within the observed feature domain (e.g., nearest-neighbour support checks). Do not optimize arbitrary synthetic feature combinations or present the output as causal, safe or regulator-approved operating advice. The dataset does not contain control interventions, constraints, permit thresholds or equipment safety envelopes.

6. Literature Review and Research Gap

Reference	What it establishes	Implication for TURBINE-X
Kochueva & Nikolskii (2021), Data Analysis and Symbolic Regression Models for Predicting CO and NOx Emissions from Gas Turbines, Computation 9(12), 139. DOI: 10.3390/computation9120139	Symbolic regression and interpretable emission modelling have already been studied.	Do not claim symbolic equations or explainability alone as novel.
Long-term atmospheric pollutant emissions from a combined cycle gas turbine: Trend monitoring and prediction applying machine learning (Fuel, 2023), DOI: 10.1016/j.fuel.2023.127722	Multi-year emissions modelling and time-shift adjustment have been investigated.	Temporal emission prediction is established.
Reliability-Oriented Predictive Emission Monitoring of Gas Turbines Under Multiyear Distribution Shift and High-Emission Tail Risk (Engineering, 2026),	Uses this same 36,733-row dataset; 2011–13 development, 2014 selection/calibration, 2015 locked test; studies tail risk and distribution shift.	Do not claim that temporal split, tail weighting or reliability monitoring alone is novel.

DOI: 10.3390/eng7100538		
Aboalhol & Abdusamad (2026), Gas turbine emissions modeling: from empirical correlations to hybrid physics-machine learning frameworks, DOI: 10.1016/j.nexres.2026.101332	Review of 56 gas-turbine emission modelling studies.	Situate conventional tabular ML against physics-based and hybrid modelling; acknowledge no physics model is available in this dataset.

6.1 Provisional gap

The defensible investigation is whether shared multi-output conventional learners can jointly estimate emissions and turbine energy yield from eight non-target measurements, and whether their per-target and simultaneous uncertainty behavior differs from independently optimized estimators under the same chronological partitions. The associated historical trade-off explorer is descriptive, not a causal optimizer. This gap is a hypothesis, not a verified claim of first-ever work. Search IEEE Xplore, Scopus, Web of Science, Crossref and Google Scholar for combinations of gas turbine, multi-task regression, multi-output regression, TEY, CO, NOx, conformal prediction and Pareto analysis before writing a novelty statement.

7. Frozen Research Questions and Hypotheses

ID	Research question / hypothesis	Test
RQ1	Does joint multi-output learning improve predictive quality versus independently fitted models?	Compare independent Ridge/ExtraTrees/CatBoost with CatBoost MultiRMSE and multi-output tree estimators.
RQ2	Are gains consistent across CO, NOX and TEY and across years?	Per-target metrics and year-specific error analysis.
RQ3	Can marginal and simultaneous intervals be calibrated on the 2014 partition?	Conformal interval coverage and width on 2015.
RQ4	Does support-aware historical trade-off analysis change candidate rankings versus mean-only ranking?	Observed/support-near candidate comparison and sensitivity analysis.
H1	Joint models may improve at least one output without unacceptable degradation in others.	Predefined multi-target score plus target-specific guardrails.
H2	Benefits of joint learning, if any, will differ by target and annual domain.	Annual metrics and paired block comparisons.
H3	Simultaneous three-output coverage will be lower than marginal coverage unless explicitly calibrated jointly.	Marginal versus joint conformal calibration.
H4	Support-restricted candidate analysis is more conservative than unconstrained feature-space optimization.	Compare candidate support distances and Pareto sets.

8. Frozen Experimental Protocol

Partition	Year(s)	Rows	Use
Development	2011–2013	22,191	Model development and expanding-window validation
Calibration/configuration	2014	7,158	Conformal calibration and prespecified configuration decisions
Locked test	2015	7,384	One-time final generalization

			evaluation

Development folds: Fold 1 trains on 2011 and validates on 2012. Fold 2 trains on 2011–2012 and validates on 2013. Hyperparameter search and AutoML use only these development folds. The selected model is refitted on the development period; 2014 is reserved for calibration/configuration as prespecified; 2015 remains locked.

This is a strict future-year evaluation. UCI recommends the first three years for training/cross-validation and the last two for testing. The 2014 calibration/selection and 2015 locked test partition also matches the published 2026 same-dataset protocol. The project's distinct question must therefore be the joint-output comparison, not a claim to have invented this split.

8.1 Leakage prevention

- Partition by annual source file before any model fitting.
- Never shuffle across years for the primary result.
- Keep the year metadata outside feature matrices.
- Do not fit preprocessing on calibration or test rows.
- Do not select hyperparameters using 2014 or 2015 labels beyond the explicitly pre-registered calibration/configuration role.
- Use 2014 labels only for the specified calibration/configuration procedure; never refit the predictive model on 2014 if the protocol treats it as calibration-only.
- Use 2015 exactly once for the final report.

9. Conventional ML Algorithm Selection

Model / method	Role	Reason / constraint
DummyRegressor	Baseline	Reference error floor; run separately per target.
Ridge / ElasticNet	Linear baselines	Fast and interpretable; scaling in fold-fitted pipeline; coefficients sensitive to collinearity.
RandomForestRegressor	Nonlinear baseline	Robust tabular ensemble; can support multi-output targets in sklearn.
ExtraTreesRegressor	Nonlinear baseline	Randomized tree ensemble; useful benchmark and relatively efficient.
CatBoostRegressor	Independent boosted model	Strong tabular learner; target-specific runs.
XGBoost	Boosting comparator	Well-established nonlinear tabular baseline; tune under fixed budget.
CatBoost MultiRMSE	Joint-output candidate	Native multi-target objective; directly tests shared-output learning.
FLAML	AutoML comparator	Bounded automated model selection per target; same temporal folds and metric.
Optuna TPE	HPO	Fixed trial budget and deterministic seeds; not itself AutoML.
Conformal prediction	Uncertainty layer	Calibrate intervals on held-out 2014; test empirical coverage in 2015.
SHAP / permutation / PDP-ALE	Explainability	Complementary explanations; correlated features require cautious interpretation.

Do not add Gaussian Processes to the main pipeline: exact GP scaling is unattractive for 36,733 rows. Do not use neural networks, deep forest, neural AutoML, or any deep-learning-based estimator. Do not include a large ensemble unless out-of-fold validation proves stable improvement.

10. Feature Engineering and Preprocessing

- Start with the 8/9 measured raw predictors; preserve original physical units for tree models.
- Standardize only for scale-sensitive linear/SVR baselines, fitting scaler within each fold.
- Do not impute current missing values; retain schema assertions. If future missingness is introduced in robustness tests, simulate it on copies and fit imputers only on training folds.
- Do not automatically clip emission tails; high values are important targets. Investigate physically implausible values separately.
- Create only justified ratios/interactions after domain review; document formula and physical interpretation before evaluation.
- Do not add time-lag or rolling features because row timestamps are unavailable.
- Use correlation/VIF diagnostics and grouped ablation rather than blind correlation-threshold deletion.
- Feature selection must be performed inside development folds only.

11. Complete ML Pipeline

9. Initialize repository, environment, seed policy and config.
10. Verify original file hashes and preserve immutable raw data.
11. Run schema, row-count, type, missingness, non-finite, duplicate and range checks.
12. Attach year from filename as metadata; preserve within-file row order.
13. Generate EDA, annual summaries, target distributions and correlation diagnostics.
14. Approve Experiment A and Experiment B feature/target manifests.
15. Create chronological development, calibration and locked-test manifests.
16. Train dummy and linear baselines.
17. Train Random Forest, Extra Trees, CatBoost, XGBoost and true multi-output models.
18. Run bounded Optuna and FLAML experiments on development folds only.
19. Compare independent models, joint models and only justified ensembles.
20. Freeze the selected model configuration.
21. Calibrate uncertainty on the 2014 partition.
22. Evaluate once on 2015, including per-target, tail, calibration and robustness metrics.
23. Perform residual, duplicate-sensitivity, feature-ablation and support analyses.
24. Create explainability artifacts and application-facing metadata.
25. Serialize final pipeline, schema and provenance.
26. Build Streamlit application, automated tests and documentation.

12. Project Folder Structure

```
TURBINE-X/
├── data/
│   ├── raw/            # Immutable five original CSVs
│   ├── interim/        # Validated concatenated data
│   ├── processed/      # Split manifests and feature matrices
│   └── external/       # No extra primary data; reference files only if licensed
├── notebooks/          # Ordered 00–17 Jupyter notebooks
├── src/
│   ├── data/           # Ingestion, schema and audit
│   ├── features/       # Feature manifests and transforms
│   ├── models/         # Baselines and conventional estimators
```

```
│   ├── evaluation/          # Metrics, calibration and tests
│   ├── visualization/       # Reusable plotting
│   └── utils/               # Config, logging, seeds, progress tracker
├── configs/                 # project.yaml, logging.yaml, tasks.json
├── experiments/             # Trial records and persistent progress state
├── outputs/
│   ├── figures/             # EDA, diagnostics, SHAP, progress chart
│   ├── tables/              # Metric and audit tables
│   └── reports/             # JSON/Markdown audit and final reports
├── models/                  # Serialized approved pipeline and metadata
├── logs/                    # Runtime logs
├── tests/                   # Unit, integration, leakage and app tests
├── app/                     # Streamlit application
├── docs/
│   ├── diagrams/            # Editable Mermaid/DOT and SVG diagrams
│   └── PROJECT_BLUEPRINT.md
├── scripts/                 # CLI entry points and progress report
├── artifacts/               # Release manifests and packaged deliverables
├── requirements.txt
├── README.md
└── .gitignore
```

13. Jupyter Notebook Execution Roadmap

Notebook	Objective	Main sections
00_environment_setup.ipynb	Environment, package versions, seed and path checks	Python/platform information; Dependency import checks; Create output directories ...
01_dataset_inventory_and_audit.ipynb	Audit all five original annual CSV files	Read raw files without mutation; Inventory file sizes, hashes, shape and schema; Validate headers and dtypes ...
02_data_understanding.ipynb	Confirm data dictionary, chronology and task formulation	Load inventory; Attach year from filename only; Validate annual row counts and order ...
03_exploratory_data_analysis.ipynb	Understand distributions, correlations and year effects	Univariate summaries; Target distributions; Annual target and feature drift ...
04_data_quality_and_cleaning.ipynb	Create a documented cleaned master table	Preserve raw inputs; Validate numeric ranges; Retain duplicates in primary data ...
05_feature_engineering.ipynb	Build the two pre-registered feature/target formulations	Experiment A: nine inputs to CO/NOX; Experiment B: eight inputs to CO/NOX/TEY; No inferred timestamp/lag features ...
06_feature_selection.ipynb	Assess redundancy and feature stability	Correlation/VIF diagnostics; Permutation importance on development folds; Grouped ablation for correlated pressure/temperature variables ...
07_train_validation_test_split.ipynb	Create immutable chronological split manifest	Development 2011-2013; Expanding folds 2011→2012 and 2011-2012→2013; 2014 calibration/selection ...
08_baseline_models.ipynb	Train dummy and regularized linear baselines	DummyRegressor; Ridge and ElasticNet pipelines; Fold-wise validation metrics ...
09_advanced_ml_models.ipynb	Compare justified conventional tree models	RandomForest and ExtraTrees; CatBoost and XGBoost; CatBoost MultiRMSE for joint output ...
10_hyperparameter_optimization.ipynb	Tune selected models with Optuna under a fixed budget	Define fold-aware objective; Fix seed and trial budget; Record all trials ...
11_automl_experiments.ipynb	Run FLAML per target as a bounded AutoML comparator	Fixed time budget; Same chronological folds; Record search space and selected pipeline ...
12_ensemble_learning.ipynb	Evaluate a small validation-selected stacking ensemble	Use out-of-fold development predictions; No test stacking; Compare ensemble against best single model ...
13_model_explainability.ipynb	Explain final candidate predictions	Permutation importance; SHAP global and local explanations; PDP/ALE for selected variables ...

14_model_evaluation.ipynb	Evaluate final regression and uncertainty metrics	MAE/RMSE/R2/MedianAE per target; Calibrate conformal intervals on 2014; Evaluate 2015 once ...
15_error_and_robustness_analysis.ipynb	Stress-test temporal and data-quality sensitivity	Residual and tail analysis; Duplicate sensitivity; Noise/missingness simulation on copies only ...
16_final_model_training_and_saving.ipynb	Freeze and serialize the selected model pipeline	Freeze model choices before test; Fit approved final model on development data; Persist model and metadata ...
17_final_results_and_reproducibility.ipynb	Create locked evaluation report and reproduction manifest	Load saved artifacts; Reproduce 2015 metrics; Create final figures and tables ...

Execution order is strictly 00 through 17. Each notebook is a reproducible stage, not a standalone source of hidden state. Promote reusable logic into src/ after a notebook experiment is validated. Save outputs to the paths declared in configs/tasks.json. Restart the kernel and run all cells before marking a notebook stage complete.

14. Centralized Automatic Progress Tracker

The starter package includes src/utils/tracker.py and configs/tasks.json. The registry stores task ID, phase, title, status, timestamps, dependencies, notebook, expected outputs, actual outputs, validation status, blockers and notes. Persistent state is written to experiments/progress_state.json.

- Allowed states: NOT_STARTED, IN_PROGRESS, BLOCKED, COMPLETED, FAILED.
- Completion requires validation_passed=True and every expected output to exist and be non-empty.
- Status changes are timestamped and persisted atomically through a temporary file replacement.
- Progress report includes overall completion, phase-wise completion, task counts, current phase, next dependency-ready task and a chart.
- Opening a notebook does not change task status. A validated output is required.
- Expected output paths must be updated if the implementation chooses different names.

```
python -m src.utils.tracker report
python -m src.utils.tracker update D01 IN_PROGRESS --notes "Started schema audit"
python -m src.utils.tracker update D01 COMPLETED --validated --notes "Inventory and schema reports checked"
```

In a notebook, import update_task from src.utils.tracker and call it only after programmatic validation. The tracker is a scaffold; test its behavior before relying on it for project governance.

15. Experiment and Output Management

Artifact	Naming convention	Required metadata
Experiment record	EXP_<YYYYMMDD>_<model>_<target>_<seed>.json	Model, target, params, seed, fold, dataset hash, package versions, runtime, metric, paths
Trial table	optuna_trials_<target>.csv	Trial ID, params, objective, fold metrics, duration, state
Model artifact	models/<experiment_id>.joblib	Pipeline, feature schema, target names, training years, code version
Figure	fig_<phase>_<topic>.png	Title, units, data subset, source, seed where relevant
Table	tbl_<phase>_<topic>.csv	Metric definition, split, target, sample count
Report	<topic>_report.md/json	Generation time, input hashes, validation state

Never overwrite a prior experiment. Use unique experiment IDs and save metadata beside artifacts. Record SHA256 hashes for the original five CSVs and the final model. Do not save 2015 labels or metrics into tuning artifacts before final model freeze.

16. Architecture Diagrams

Fifteen editable diagrams are included in docs/diagrams as Mermaid (.mmd), Graphviz (.gv) and SVG files. They cover the system architecture, workflow, ingestion, preprocessing, EDA, feature engineering, split/leakage control, model training, AutoML, optional ensemble, evaluation, XAI, application, progress tracking and roadmap.

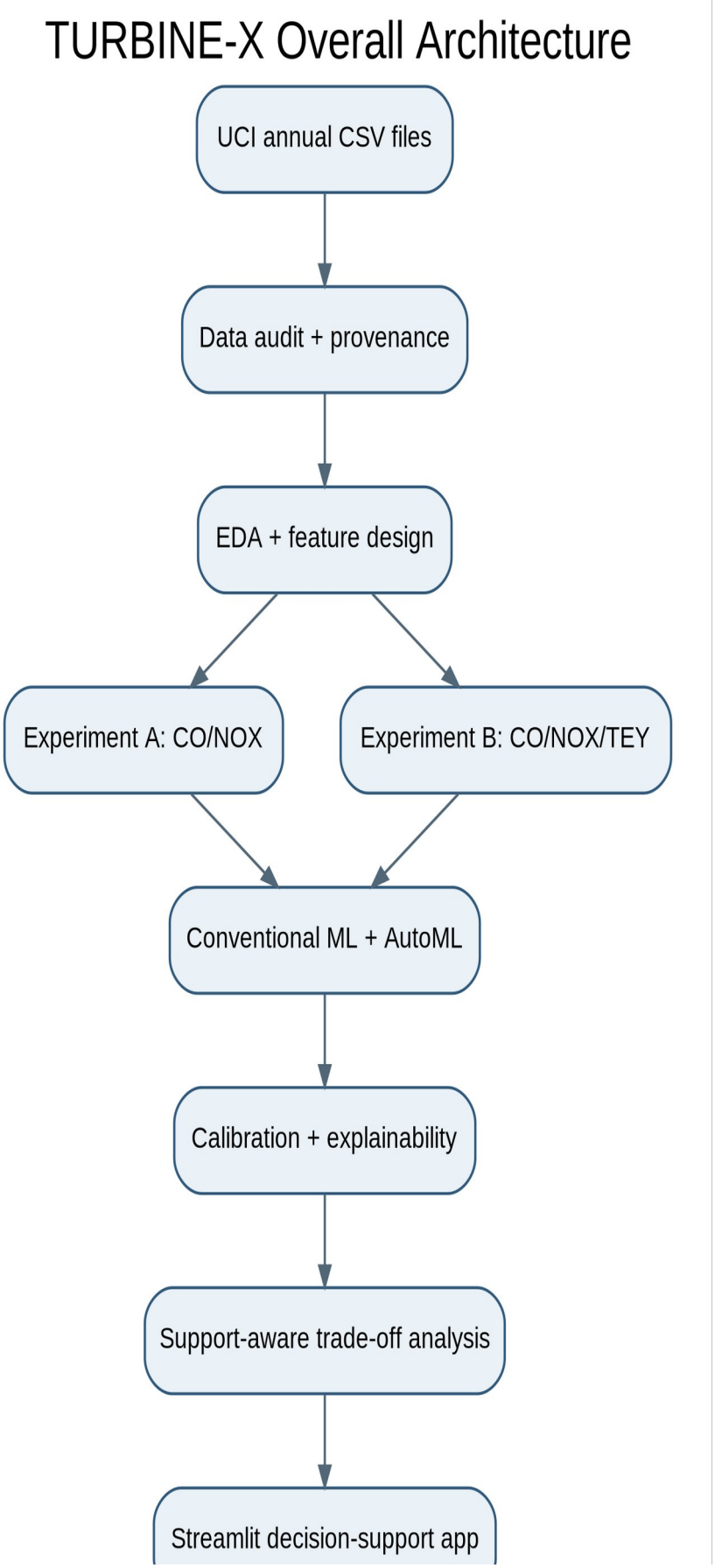

TURBINE-X Overall Architecture
UCI annual CSV files
Data audit + provenance
EDA + feature design
Experiment A: CO/NOX
Experiment B: CO/NOX/TEY
Conventional ML + AutoML
Calibration + explainability
Support-aware trade-off analysis
Streamlit decision-support app

Figure 1. TURBINE-X Overall Architecture

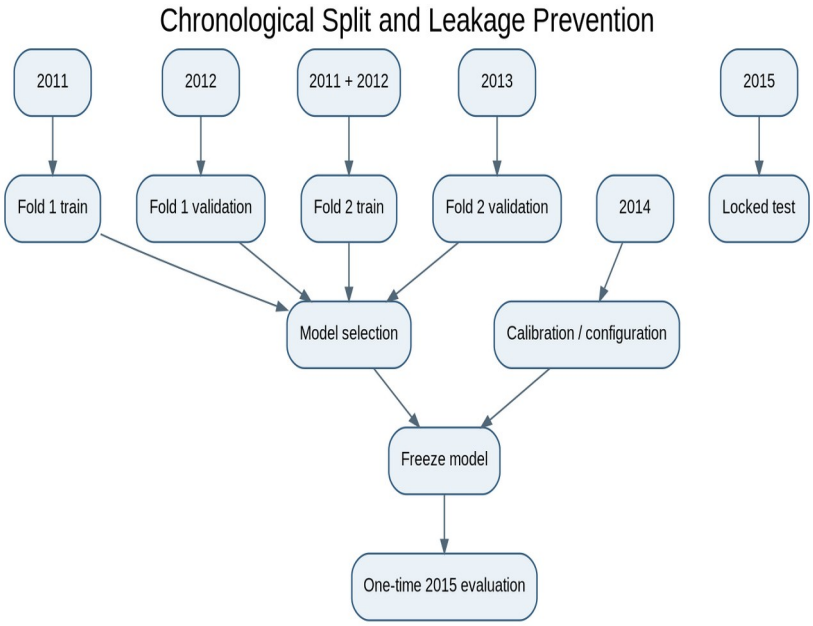


Figure 7. Chronological Split and Leakage Prevention

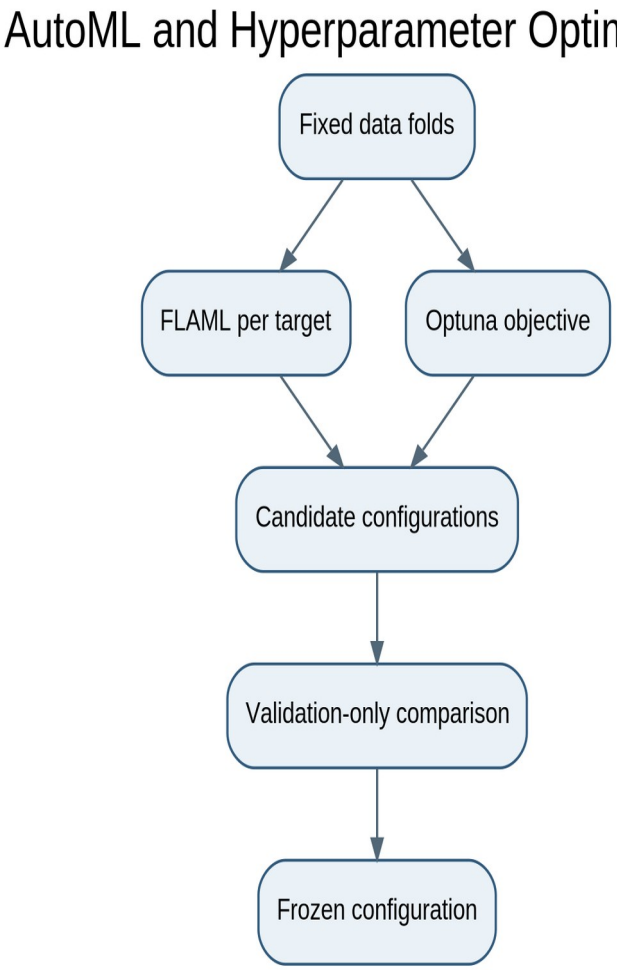


Figure 9. AutoML and Hyperparameter Optimization

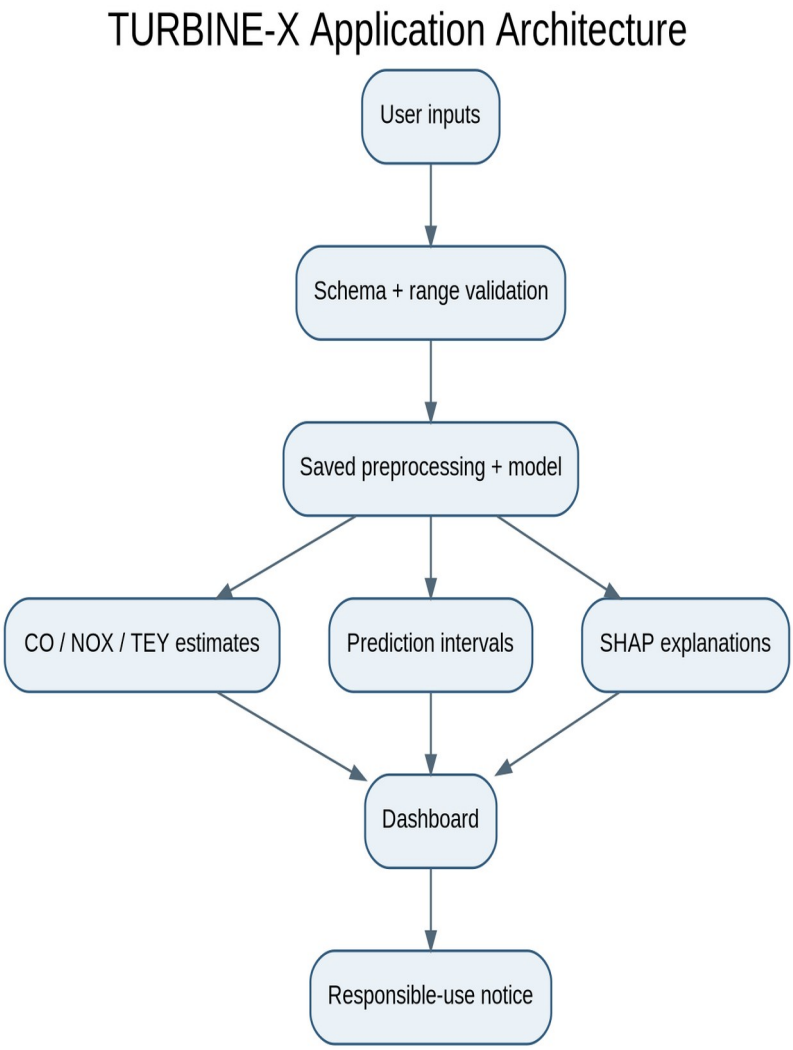


Figure 13. TURBINE-X Application Architecture

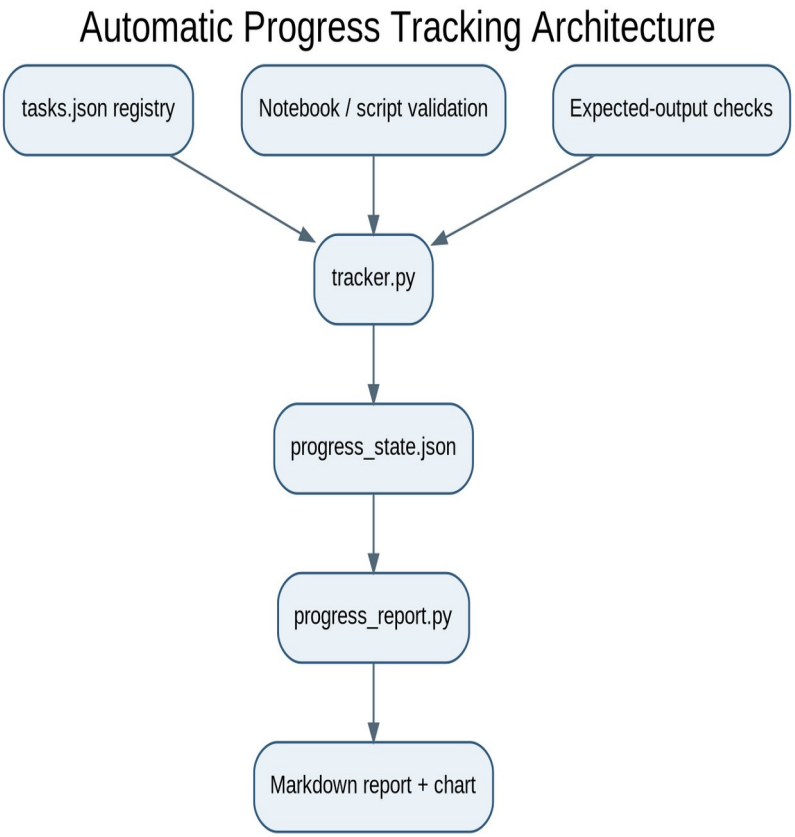


Figure 14. Automatic Progress Tracking Architecture

17. Week-by-Week Implementation Schedule

Week	Objective	Notebook	Expected outputs	Validation / completion	Common mistake
1	Dataset audit	01_dataset_inventory_and_audit.ipynb	Inventory, schema, quality JSON, hashes	Reconcile all five row counts/headers; zero missing/nonfinite; duplicates documented	Do not concatenate before schema agreement
2	EDA and problem freeze	02_data_understanding.ipynb; 03_exploratory_data_analysis.ipynb	Data dictionary, EDA figures, problem definition	Annual totals and feature/target summaries match raw files	No fabricated timestamps or lag features
3	Cleaning and features	04_data_quality_and_cleaning.ipynb; 05_feature_engineering.ipynb	Clean master, duplicate sensitivity copy, feature manifest	Assert exact columns, target isolation and row preservation	Do not silently drop duplicate records
4	Feature diagnostics and splits	06_feature_selection.ipynb; 07_train_validation_test_split.ipynb	Feature diagnostics, immutable split manifest	No year overlap; train folds strictly precede validation	Never random-split primary result
5	Baselines	08_baseline_models.ipynb	Dummy/Ridge/ElasticNet metrics	Fold metrics and reproducible pipeline	Scaling fit inside fold
6	Advanced models	09_advanced_ml_models.ipynb	RF/ExtraTrees/ CatBoost/XGBoost and joint model metrics	Independent vs joint models compared on identical folds	Do not select on test
7	HPO and AutoML	10_hyperparameter_optimization.ipynb; 11_automl_experiments.ipynb	Optuna trials, FLAML summary	Fixed budget, objective and folds logged	Avoid runaway parallel jobs on 16 GB RAM
8	Ensemble and XAI	12_ensemble_learning.ipynb;	Ensemble comparison, SHAP and PDP/ALE	Keep ensemble only if stable validation	Correlated-feature explanations are not

		13_model_explainability.ipynb	figures	improvement	causal
9	Uncertainty and final evaluation	14_model_evaluation.ipynb	2014 calibration and 2015 locked metrics	Coverage, width and all target metrics reported	Do not recalibrate on 2015
10	Robustness	15_error_and_robustness_analysis.ipynb	Residual/tail/ duplicate/support reports	Sensitivity results and limitations documented	Synthetic missingness only on copies
11	Final model and app	16_final_model_training_and_saving.ipynb; app/app.py	Model artifact, metadata, dashboard	Input schema and prediction smoke tests pass	Do not claim control or safe setpoints
12	QA and reporting	17_final_results_and_reproducibility.ipynb	Final report, tests, DOCX/PDF research report	Re-run from clean environment; test report saved	All claims traceable to artifacts

18. Evaluation and Statistical Analysis

18.1 Regression metrics

- MAE and RMSE for CO, NOX and TEY separately.
- R^2 per target; do not aggregate R^2 across targets.
- Median absolute error for robustness to extreme residuals.
- Normalized RMSE only with a clearly stated normalization denominator derived from development data.
- Report annual/test sample count and target range alongside every metric.

18.2 Uncertainty metrics

- Marginal 90% and 95% interval coverage and mean interval width per target.
- Simultaneous coverage: proportion of rows for which all three outputs are covered.
- Conformal calibration quantiles estimated from 2014 only.
- Report empirical 2015 coverage; do not imply distribution-free guarantees under arbitrary temporal shift.

18.3 Statistical analysis

- Paired moving-block bootstrap for differences between model errors; report block length sensitivity.
- Use identical test observations for paired model comparisons.
- Correct for multiple comparisons if many models/targets are tested.
- Use ablations for joint versus independent outputs, feature groups and duplicate handling.
- Do not treat every adjacent row as independent or interpret block length in hours without timestamps.

19. Testing and Quality Assurance

Test category	Test examples	Automation
Dataset integrity	File exists, SHA256, expected row counts, header order	pytest
Schema	11 expected float columns, no unexpected fields	pytest
Missing/finite	No NA/NaN/inf in original snapshot	pytest
Duplicates	Expected per-file counts and cross-year overlap	pytest
Feature validity	Experiment A/B columns exact; target never in X	pytest
Leakage	Year partitions disjoint; train year < validation year	pytest
Preprocessing	Fit only on fold training indices	Pipeline tests

Metrics	Known arrays produce expected MAE/RMSE/R^2	pytest
Model I/O	Serialized model loads and predicts expected output shape	pytest
Application	Invalid input rejected; valid input returns all target outputs	Streamlit smoke test
Tracker	Unique IDs, dependency readiness, output existence gate	pytest
Reproducibility	Seed, package versions and data hashes captured	CI/manual release check

19.1 Suggested tests directory

tests/\n├── test_schema.py\n├── test_integrity.py\n├── test_duplicates.py\n├── test_feature_manifest.py\n├── test_split_leakage.py\n├── test_preprocessing.py\n├── test_metrics.py\n├── test_model_serialization.py\n├── test_tracker.py\n└── test_app_smoke.py

20. Application Design

Build a Streamlit demonstration only after the final pipeline is frozen. Pages: (1) Overview and model scope; (2) Input validation; (3) CO/NOX/TEY estimates; (4) marginal and simultaneous prediction intervals; (5) SHAP explanation for supported model types; (6) observed/support-near historical operating-point comparison; (7) model limitations and data provenance.

- Validate numeric inputs, allowed schema and broad observed support ranges.
- Warn when input is far from the development data distribution.
- Show units and clearly label predictions versus measured historical records.
- Do not present model output as a safe setpoint, causal intervention or compliance result.
- Store model version, feature order, target order and training partition metadata.

21. Hardware, Software and Environment

Resource	Plan
CPU	AMD Ryzen 9 8940HX; CPU-first training
GPU	RTX 5060 8 GB; optional, not required
RAM	16 GB; cap parallel jobs and avoid large nested CV
Storage	1 TB SSD; raw data under 3 MB total, model artifacts and outputs modest
OS / IDE	Windows 11, VS Code, Jupyter
Python	Python 3.11 recommended; create isolated .venv
Core packages	pandas, NumPy, SciPy, scikit-learn, CatBoost, XGBoost, FLAML, Optuna, SHAP, MAPIE, Streamlit, matplotlib, Plotly, pytest, PyYAML

22. Risk Register and Mitigations

Risk	Severity	Mitigation
Existing 2026 paper uses same dataset and temporal split	High	Novelty must focus on joint CO/NOX/TEY outputs; conduct systematic literature search before claiming novelty.
Single plant limits external validity	High	State single-site limitation; no cross-plant

		generalization claim.
No timestamps	High	No lags, rolling features or lead-time forecasting; year-level temporal evaluation only.
Correlated operating features	Medium	Grouped ablation, stability analysis, cautious SHAP interpretation.
TEY leakage in joint task	High	TEY excluded from Experiment B predictors; assert feature/target disjointness.
Temporal target shift	High	Chronological holdout, per-year errors, uncertainty coverage reporting.
Observational Pareto results mistaken for causal control	High	Restrict to observed/support-near points; explicit non-causal disclaimer.
Overfitting AutoML search	Medium	Fixed trial/time budget and development-only validation.
16 GB RAM exhaustion	Medium	Limit n_jobs, avoid excessive parallel trials, use compact outputs.
Tracker false completion	Medium	Require validation flag and non-empty expected output files.

23. Expected Deliverables and Acceptance Criteria

Deliverable	Acceptance criterion	Status
Dataset audit report	Five original files audited with actual hashes, rows, schema, missingness and duplicates	Completed in this session; machine-readable report in starter package
Research methodology	Two clear formulations; no leakage; chronological protocol; limitations	Provisional freeze included
Project starter package	Folder structure, config, notebook templates, tracker, tests, diagrams, raw CSVs	Created
Progress tracker	Persists state; output gate; report and chart generation	Code included; requires local test run
Diagrams	15 editable Mermaid/DOT and SVG diagrams	Created; inspect/adjust to final implementation
DOCX blueprint	Professional sections, audit tables, roadmap and diagrams	Generated
PDF blueprint	Readable exported copy	Generated and checked for existence
Trained ML models	Reproducible metrics and saved model	Not started
Working Streamlit app	Validated model input/output and tests	Not started
Research paper	Novelty and results supported by actual experiments	Not started

24. Final Supervisor Verdict

The uploaded files are suitable for a serious conventional-ML regression study. They are not suitable for causal optimization, safe turbine control or exact time-series forecasting because the data are observational, single-site and lack row-level timestamps and engineering constraints. The most defensible project is a controlled study of independent versus joint multi-output learning for CO, NOX and TEY, with uncertainty and historical trade-off analysis as supporting components.

The project should proceed only after the student confirms the two formulations and accepts the limitations. The starter package is scaffolding; model code and application implementation remain future work.

25. References

27. UCI Machine Learning Repository. Gas Turbine CO and NOx Emission Data Set (2019). DOI: https://doi.org/10.24432/C5WC95. https://archive.ics.uci.edu/dataset/551/gas+turbine+co
28. Kochueva, O., & Nikolskii, K. (2021). Data Analysis and Symbolic Regression Models for Predicting CO and NOx Emissions from Gas Turbines. Computation, 9(12), 139. https://doi.org/10.3390/computation9120139
29. Long-term atmospheric pollutant emissions from a combined cycle gas turbine: Trend monitoring and prediction applying machine learning. Fuel (2023), Article 127722. https://doi.org/10.1016/j.fuel.2023.127722
30. Rodríguez-Abreo, O., et al. (2026). Reliability-Oriented Predictive Emission Monitoring of Gas Turbines Under Multiyear Distribution Shift and High-Emission Tail Risk. Engineering, 7(10), 538. https://doi.org/10.3390/eng7100538
31. Aboalhol, T. R., & Abdusamad, K. A. (2026). Gas turbine emissions modeling: from empirical correlations to hybrid physics-machine learning frameworks. DOI: https://doi.org/10.1016/j.nexres.2026.101332
32. Vovk, V., Gammerman, A., & Shafer, G. (2005). Algorithmic Learning in a Random World. Springer.
33. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS.
34. Prokhorenkova, L., et al. (2018). CatBoost: unbiased boosting with categorical features. NeurIPS.
35. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.